

SISTEMA DE VENTAS

Documentación técnica: Sistema de Base de Datos Distribuida

- Entorno: PostgreSQL 17
- SO: Windows 10

Descripción del sistema

El proyecto consiste en la creación de un sistema de información de una cadena de tiendas tipo Office Depot. Utilizando **PostgreSQL 17** instalado en **Windows 10** se hará la simulación en una sola computadora mediante dos instancias de PostgreSQL.

- Distribución (**SiteA: puerto 5233 y SiteB: puerto 5234**).

El objetivo es garantizar **alta disponibilidad (24x7)**, **redundancia**, **distribución de carga** y **recuperación ante fallos**, implementando **replicación física (streaming)**, **consultas distribuidas (postgres_fdw)** y **estrategias de respaldo**.

La replicación física o de streaming: es una función estándar de PostgreSQL, la cual permite transferir la información actualizada del servidor en espera en tiempo real, de modo que las bases de datos de ambos servidores se mantengan sincronizadas. La replicación física es utilizada porque aporta beneficios en el sistema, por ejemplo, cuando falla el servidor principal, el servidor en espera puede hacerse cargo de la operación. También el procesamiento de SQL de solo lectura se puede distribuir entre varios servidores.

Las consultas distribuidas con PostgreSQL Foreign Data Wrappers (FDW): permiten ejecutar consultas SQL en múltiples bases de datos como si fueran tablas locales. FDW accede a datos externos sin necesidad de replicación, realizando operaciones como JOINS entre tablas remotas y locales, y puede optimizar las consultas enviando filtros al servidor remoto para ejecutarlos allí, y así reducir la transferencia de datos. Otra ventaja del FDW es que se pueden definir permisos por servidor o por tabla, lo que permite tener una seguridad más controlada en el sistema.

1. ESTRUCTURA DEL SISTEMA DISTRIBUIDO

Creación de dos instancias de la base de datos (SIN USAR VIRTUALIZACION) para la simulación de la base de datos distribuida. Las instancias generadas son desarrolladas por la versión PostgreSQL 17.

- Creación de carpetas: (abre PowerShell como administrador y ejecuta)

```
mkdir "C:\PostgreSQL\siteA\data"
```

```
mkdir "C:\PostgreSQL\siteB\data"
```

```
mkdir "C:\PostgreSQL\wal_archive"
```

- Inicialización de instancias desde el directorio (CMD como administrador)

```
cd C:\Programa Files\PostgreSQL\17\bin (o de donde este initdb.exe)
```

```
initdb -D "C:\PostgreSQL\siteA\data" -U postgres -W
```

```
initdb -D "C:\PostgreSQL\siteB\data" -U postgres -W
```

Te pedirá una contraseña para el “superusuario”: (Ejemplo: postgres)

2. CONFIGURACIÓN DE INTANCIAS

- Configuración de instancias escuchada por puertos diferentes, pero en la misma computadora.
Resultado:

Sitio	SiteA	SiteB
Puerto	5233	5234
Carpeta de datos	C:\PostgreSQL\siteA\data	C:\PostgreSQL\siteB\data
Rol principal	Primary / Master	Standby / Replica

Ambos nodos están alojados en la misma computadora física y se comunican a través de localhost.

- Configuración archivos postgresql.conf (Editar con un procesador de texto).
 - Nota: Es necesario eliminar el signo “#” del inicio de cada configuracion para que sean aplicados en las instancias.

Para SiteA: C:\PostgreSQL\siteA\data\postgresql.conf

CONECTIVIDAD Y PUERTOS	
listen_addresses = '*'	Le dice al servidor que escuche conexiones en todas las interfaces de red disponibles, no solo en la interfaz local (localhost). Esto es esencial para que tus dos instancias simuladas (Sitio 1 y Sitio 2) puedan comunicarse y para que otros clientes (la aplicación) puedan conectarse.
port = 5233	Especifica el puerto TCP/IP que la instancia usará para las conexiones. En un sistema distribuido simulado en una misma máquina, debes usar puertos diferentes (ej. 5233 para la Maestra y 5234 para la Réplica) para evitar conflictos.
REPLICA Y ALTA DISPONIBILIDAD	
wal_level = replica	Incrementa la cantidad de información que se escribe en los WAL (Write-Ahead Logs) . El valor replica es el mínimo necesario para que otras instancias puedan usar el WAL para replicar datos y para realizar PITR.
max_wal_senders = 10	Define el número máximo de procesos (wal senders) que pueden enviar datos WAL a las instancias réplicas. Un valor de 10 permite que hasta 10 servidores de réplica se conecten simultáneamente.
max_replication_slots = 10	Define el número máximo de Slots de Replicación que se pueden crear. Los <i>slots</i> aseguran que el servidor primario no elimine los archivos WAL hasta que la réplica (o réplicas) haya confirmado recibirlos. Esto es fundamental para la estabilidad de la replicación.
synchronous_commit = on	Controla cuándo una transacción es considerada exitosa. El valor on (o un valor específico de réplica, como remote_write) obliga a la Maestra a esperar la confirmación de la Réplica de que los datos han sido escritos de forma segura en sus logs. Esto reduce el rendimiento, pero garantiza que la Réplica esté sincronizada y evita la pérdida de transacciones críticas durante un <i>failover</i> .
RESPALDO Y RECUPERACIÓN (PITR)	
consiste en el copiado y almacenamiento de los archivos de registro de escritura anticipada (WAL) a una ubicación externa a medida que se generan, lo que permite realizar recuperaciones puntuales (Point-in-Time Recovery o PITR) a cualquier momento, ya que asegura la disponibilidad de la secuencia completa de transacciones.	

archive_mode = on	Habilita el modo de archivado. Esto hace que, una vez que un archivo WAL esté completo, PostgreSQL ejecute el comando definido en archive_command para moverlo a una ubicación de almacenamiento a largo plazo (el archivo WAL).
archive_command = 'copy "%p" "C:\\PostgreSQL\\wal_archive\\%f"'	Define el comando de shell que se ejecutará para copiar cada archivo WAL completado. Es la instrucción real que mueve los archivos WAL (%p es la ruta del archivo, %f es el nombre del archivo) a un directorio de respaldo seguro. Almacenar estos archivos es lo que permite restaurar la base de datos a cualquier momento exacto en el pasado.

Para SiteB: C:\\PostgreSQL\\siteB\\data\\postgresql.conf

listen_addresses = '*'	
port = 5234	
wal_level = replica	
max_wal_senders = 10	
hot_standby = on	Permitir Consultas de Lectura: Habilita la instancia de réplica para aceptar y procesar consultas de solo lectura (<i>read-only queries</i>) mientras la base de datos se encuentra en modo de recuperación (es decir, recibiendo datos del servidor primario/maestro). Esto permite que la réplica sea utilizada para balancear la carga de trabajo de lectura, sirviendo a reportes o aplicaciones que no requieren escribir datos. Preparación para la Promoción: Es necesario para que la instancia de réplica pueda ser promovida a servidor primario (<i>failover</i>) en caso de que el servidor maestro falle. Cuando una réplica tiene hot_standby = on, puede reanudar la operativa normal de lectura/escritura cuando se activa la promoción.
archive_mode = on	
archive_command = 'copy "%p" "C:\\PostgreSQL\\wal_archive\\%f"'	

- Configurar pg_hba (Editar con un procesador de texto): Esta configuración debe ser realizada en ambas instancias.

Para SiteA: (C:\PostgreSQL\siteA\data\pg_hba.conf)

Para SiteB: (C:\PostgreSQL\siteB\data\pg_hba.conf)

# IPv4 local connections: host all all 0.0.0.0/0 md5	Esta regla permite a los usuarios regulares conectarse a cualquiera de las bases de datos de la instancia (SiteA o SiteB) desde la propia máquina local
# replication privilege. host replication all 0.0.0.0/0 md5	Esta regla es crítica para configurar el clúster distribuido y la replicación entre las dos instancias: permite que la Instancia Réplica se conecte al usuario especial de replicación (repl_user) de la Maestra para que pueda recibir los cambios de la base de datos.

3. CLÚSTER DE POSTGRES (STREAMING REPLICATION)

Conexión de las 2 instancias en un clúster de postgres .

- Creación de Usuario de replicación: (Abrir psql en puerto 5233 que corresponde a SiteA y ejecute)

```
CREATE ROLE repl_user WITH REPLICATION LOGIN PASSWORD 'replica123';
```

- Inicio de instancias (abrir dos ventanas de cmd como administrador)

Para SiteA: (abrir cmd como administrador)

```
"C:\Program Files\PostgreSQL\17\bin\pg_ctl" -D "C:\PostgreSQL\siteA\data" -l "C:\PostgreSQL\siteA\log.txt" start
```

“Si el SiteA se inició correctamente dará un aviso “server started” o “start”

Para SiteB: (abrir cmd como administrador)

```
"C:\Program Files\PostgreSQL\17\bin\pg_ctl" -D "C:\PostgreSQL\siteB\data" -l "C:\PostgreSQL\siteB\log.txt" start
```

“Si el SiteB se inició correctamente te dará un aviso “servidor iniciado” o “start”

- Creación de replicación (streaming replication)
 1. Detenemos la instancia SiteB: (cmd como administrador)

```
"C:\Program Files\PostgreSQL\17\bin\pg_ctl" -D "C:\PostgreSQL\siteB\data" stop
```

“Si el SiteB se pauso correctamente te dará un aviso “servidor detenido”

2. Borra su contenido de SiteB excepto la carpeta: (cmd como administrador)

```
rmdir /S /Q "C:\PostgreSQL\SiteB\data"
```

```
mkdir "C:\PostgreSQL\SiteB\data"
```

3. Hacemos el backup desde SiteA:

```
"C:\Program Files\PostgreSQL\17\bin\pg_basebackup.exe" -h localhost -p 5234 -D
```

```
"C:\PostgreSQL\SiteB\data" -U repl_user -P -R
```

- Realiza una copia base del servidor SiteA y la coloca en el directorio de datos de SiteB.
- **Parámetros clave:**
- -h localhost: conecta al host local.
- -p 5234: puerto de SiteA.
- -D: destino del backup (SiteB).
- -U repl_user: usuario de replicación.
- -P: muestra progreso.
- -R: crea automáticamente el archivo standby.signal y configura primary_conninfo para que SiteB se conecte a SiteA como réplica.

“Si el SiteB se replica correctamente te dará un aviso (24131/24131 kB (100%), 1/1 tablespace)”

4. Reiniciamos o volvemos a Inicial la instancia SiteB:

```
"C:\Program Files\PostgreSQL\17\bin\pg_ctl.exe" -D "C:\PostgreSQL\SiteB\data" -l
```

```
"C:\PostgreSQL\SiteB\log.txt" start
```

“Si el SiteB se inició correctamente te dará un aviso “servidor iniciado” o “start” y comienza un checkpoint:time”

5. Verificamos que la replicación funciona: (abrimos psql)

```
SELECT pid, state, sync_state, client_addr FROM pg_stat_replication;
```

4. CRITERIOS DE DISTRIBUCIÓN

Criterios de distribución de la base de datos.

Tipo de distribución: **Replicación maestro-esclavo (streaming replication)**

- La instancia maestra “SiteA” recibe todos los cambios y las réplicas “SiteB” son de solo lectura.
- Permite consultas de reporte y alta disponibilidad.

El modelo lógico contiene tablas de:

- **Tiendas, Empleados, Clientes, Productos, Categoría_productos, Proveedores, Inventario, Venta, Detalles_venta, Compras, Facturacion, Puesto_empleados,**

Criterio de distribución:

- Tablas maestras y de catálogo (productos, categorías, proveedores) se **replican completas** en ambos sitios.
- **Tiendas:** pueden estar “localizadas” en nodos distintos.
- **Inventario:** depende de tienda, por lo que conviene almacenar inventario **por nodo de tienda**.
- **Venta y detalles_venta:** se pueden guardar en el nodo donde está la tienda que realiza la venta.
- **Productos y proveedores:** se pueden replicar a todos los nodos (read-only para evitar inconsistencias).
- Idea:

Nodo	A	→	Tiendas	1–5
Nodo	B	→	Tiendas	6–10

Productos/Proveedores → replicados en todos los nodos
- Esto minimiza la necesidad de locks distribuidos.

5. RESPALDO

Tipos, estrategias y modos de respaldos.

1. Tipos de Respaldo y Frecuencia

Tipo	Descripción	Ventajas	Frecuencia sugerida
Respaldo lógico (pg_dump / pg_dumpall)	Exporta la estructura y datos a un archivo .sql o .tar	Flexibilidad y Portabilidad. Útil para restauración de objetos específicos (estructura o datos) o migración de esquemas.	Diario o semanal
Respaldo físico (pg_basebackup o copia del cluster)	Copia byte a byte del directorio de datos	Base de Restauración Rápida. Rápido para restaurar servidores completos	Semanal o antes de actualizaciones
Respaldo continuo (WAL Archiving)	Guarda cada transacción en archivos WAL	Máxima Protección de Datos. Permite recuperación punto-en-el-tiempo (PITR)	Permanente (24/7)

2. Estrategias y Modos (HA y PITR)

A. Modo de Alta Disponibilidad (HA) y Redundancia

Esta estrategia se cumple con la Replicación Física (Streaming Replication) en SiteA y SiteB.

Estrategia: El Sitio A (Maestro) transmite continuamente sus archivos WAL al Sitio B (Réplica), manteniendo al Sitio B casi en tiempo real con el Maestro.

Utilizamos una configuración Síncrona (`synchronous_commit = on`) para transacciones críticas, asegurando que una transacción solo se considere cuando la réplica ha confirmado que recibió los datos. Esto previene la pérdida de transacciones críticas en caso de falla repentina del Maestro.

Para el propósito del sitio 24x7: Si el Sitio A falla, el Sitio B se puede promover a Maestro rápidamente, minimizando el tiempo de inactividad.

Verificación de Failover (Promoción de SiteB a Maestro)

El proceso de promoción a Maestro se verifica comprobando si el servidor ya no está en modo de recuperación y si puede ejecutar operaciones de escritura (INSERT/UPDATE/DELETE).

Requisito Previo: (Haberlo promovido)

```
"C:\PostgreSQL\bin\pg_ctl" -D "C:\PostgreSQL\siteB\data" promote
```

Comandos de Verificación

Una vez promovido, conecta psql al Sitio B y realiza estas consultas:

1.- Verificar el Estado de Recuperación (Es el más rápido)

Una réplica siempre estará en "modo de recuperación". Un maestro no lo está.

-- Conéctate al Sitio B (ahora Maestro)

```
psql -U postgres -p 5234
```

-- Ejecuta la siguiente consulta:

```
SELECT pg_is_in_recovery();
```

Resultado Esperado: f (false)

Significado: Confirma que el servidor ha salido del modo "Standby" y está operando como un servidor principal completamente funcional.

2.- Verificar la Función del Servidor (Identificación del Rol)

Esta función te indica la función actual del servidor.

```
SELECT pg_is_in_backup(), pg_is_in_recovery();
```

Resultado Esperado: f, f

Significado: Confirma que no está en un proceso de respaldo y que no está en modo de recuperación.

3.- Prueba de Escritura (Evidencia de Control Total)

La prueba definitiva de que el Sitio B es el nuevo Maestro es intentar una operación de escritura (que las réplicas en modo hot_standby no permiten).

```
CREATE TABLE IF NOT EXISTS evidencia_failover (
```

```
    id serial primary key,
```

```
    evento text
```

```
);
```

```
INSERT INTO evidencia_failover (evento) VALUES ('Sitio B promovido a Maestro.');
```

```
SELECT * FROM evidencia_failover;
```

Resultado Esperado: El comando CREATE TABLE se ejecuta sin error (si estuviera en modo réplica, daría el error "cannot execute CREATE TABLE in a read-only transaction").

Restableciendo la Alta Disponibilidad (Reintegración del Sitio A)

Paso 1: Detener Completamente el Sitio A (Doble Chequeo)

```
"C:\PostgreSQL\bin\pg_ctl" -D "C:\PostgreSQL\siteA\data" stop
```

Paso 2: Borrar el Contenido Antiguo (Si no lo hiciste con rmdir /S /Q)

(Si borraste el contenido del Sitio A con un comando de Windows (como rd o rmdir), este paso ya está hecho. Si no, elimina los datos antiguos de la instancia fallida para prepararla para la nueva base de datos.)

Borrar el directorio data del Sitio A (Usando la sintaxis que te funciona en tu consola):

```
rd /S /Q "C:\PostgreSQL\siteA\data"
```

Volver a crear la carpeta data vacía:

```
mkdir "C:\PostgreSQL\siteA\data"
```

Paso 3: Tomar el Base Backup (Sitio B -> Sitio A)

Ahora usamos el Sitio B (el nuevo Maestro en el puerto 5234) para crear la copia de base de datos para el Sitio A. El Sitio A se convierte en la réplica del Sitio B.

Tomar Backup desde el Nuevo Maestro (Sitio B):

```
"C:\PostgreSQL\bin\pg_basebackup.exe" -h localhost -p 5234 -D "C:\PostgreSQL\siteA\data" -U repl_user  
-P -R
```

Paso 4: Iniciar el Sitio A como Nueva Réplica

Iniciar Sitio A:

```
"C:\PostgreSQL\bin\pg_ctl.exe" -D "C:\PostgreSQL\siteA\data" -l "C:\PostgreSQL\siteA\log.txt" start
```

Resultado esperado: Aviso de "servidor iniciado".

Verificación Final (Documentación)

Conéctate al Sitio B (Maestro) y verifica que el Sitio A ahora esté en modo Réplica (Standby):

```
psql -U postgres -p 5234
```

```
-- Una vez conectado, ejecuta en el Sitio B (Maestro): SELECT client_addr, state, sync_state FROM pg_stat_replication;
```

Resultado Esperado: state debe ser streaming o similar. Esto prueba que la HA ha sido restablecida y ambos sitios están listos.

B. Modo de Recuperación ante Desastres (PITR)

Esta estrategia se basa en el Respaldo Continuo (WAL Archiving).

Estrategia: Configuramos `archive_mode = on` y un `archive_command`, esto para guardar automáticamente todos los archivos WAL en una ubicación de almacenamiento a largo plazo, fuera de los directorios de datos principales.

Para el propósito del sitio 24x7: Esto nos permite alcanzar un RPO (Recovery Point Objective) muy bajo (cercano a cero), ya que podemos reconstruir el estado de la base de datos en cualquier segundo ante del fallo, utilizando el último respaldo físico y reproduciendo los WALs archivados.

3. Recuperación de Transacciones en Caso de Falla

Falla del Servidor Maestro (Sitio A)	Las transacciones ya se habían escrito en el log del Sitio B (Réplica) debido al modo síncrono. El Sitio B se promueve a Maestro y el sistema continúa operando. No hay pérdida de transacciones que ya se habían confirmado.
Corrupción de Datos o Error Humano	Se usa la PITR. Se toman el último respaldo físico y la secuencia completa de archivos WAL archivados. Se recupera la base de datos hasta un punto en el tiempo justo antes de que ocurriera el error, recuperando todas las transacciones legítimas.
Falla de Sitio Completo	Se restaura la base de datos utilizando la copia del Respaldo Físico y los WAL almacenados,

	garantizando la recuperación de las transacciones hasta el momento del fallo.
--	---

Procedimiento de Recuperación de Transacciones Críticas (PITR)

Paso Documentado	Comando Clave de Postgres	Objetivo
A. Inicialización del Directorio	Detener el servidor y eliminar/limpiar el directorio de datos corrupto.	Preparar el sitio para la restauración.
B. Restaurar el Último Respaldo	pg_basebackup (o restaurar la copia de la data directory).	Reinstalar la estructura de la base de datos a un punto conocido (el momento del último Base Backup semanal).
C. Configurar el Punto de Recuperación	Crear un archivo recovery.signal con el archivo standby.signal (en versiones antiguas) o con el archivo de configuración.	Indicar al servidor que, al iniciar, no debe operar normalmente, sino entrar en modo. Recuperación y avanzar solo hasta una marca de tiempo específica.
D. Iniciar la Recuperación	pg_ctl start	El servidor lee el archivo recovery.signal, busca los WALs en el wal_archive (tu carpeta), y los aplica secuencialmente hasta el momento deseado, recuperando todas las transacciones perdidas.